

# MÉTODO PARA DESENVOLVIMENTO DE SISTEMAS ORIENTADOS A REGRAS UTILIZANDO O PARADIGMA ORIENTADO A NOTIFICAÇÕES

Igor T. M. Mendonça<sup>1,2,3</sup>; Jean M. Simão<sup>1,2</sup>; Luciana V. B. Wiecheteck; Paulo C. Stadzisz<sup>1,2</sup>

1 – Programa de Pós-Graduação em Engenharia Elétrica e Informática Industrial (CPGEI/UTFPR)

2 – Universidade Tecnológica Federal do Paraná (UTFPR) – Curitiba – PR, Brasil (41 33104760)

3 – Instituto Federal de Santa Catarina (IFSC) – Florianópolis – SC, Brasil

igor@ifsc.edu.br, jeansimao@utfpr.edu.br, lucianavbw@gmail.com, stadzisz@utfpr.edu.br

## Introdução

O desenvolvimento de sistemas de Inteligência Artificial e Inteligência Computacional, notadamente sistemas especialistas e *fuzzy*, utilizam, com frequência, Sistemas Orientados a Regras (SOR). Tradicionalmente em SOR são empregados Sistemas Baseados em Regras (SBR) que usam motores de inferência para associar fatos e regras e cadenciar a execução dos sistemas. Como alternativa, no contexto de SOR, o Paradigma Orientado a Notificações (PON) aparece como resposta, tanto para sistemas *crisp* quanto *fuzzy*, para superar algumas deficiências encontradas em SBRs e, mesmo, em outros paradigmas de programação atuais. Entre estas deficiências, está, principalmente, a necessidade de motores de inferência para conduzir o fluxo lógico do software.

## Objetivos

Propor um método para projetos de software que usam o PON em seu desenvolvimento. O método proposto, chamado Desenvolvimento Orientado a Notificações (DON), abrange as fases de requisitos e projeto de software para o PON.

## O Paradigma Orientado a Notificações

O PON é baseado em entidades pequenas, inteligentes e desacopladas que colaboram por meio de notificações precisas para realizar a inferência de software. Isto permite melhorar o desempenho do software e, potencialmente, torna mais fácil sua composição, tanto dos distribuídos quanto dos não-distribuídos.

As entidades que compõem o PON são ilustradas na Fig. 1, por meio de seu metamodelo. Diferente de outras abordagens nas quais é necessário um motor de inferência, no PON a inferência é realizada pela notificação pontual entre as entidades da aplicação. No PON as entidades possuem certa reatividade, permitindo que elas, ao detectar a modificação no seu estado, notifiquem outras entidades que têm interesse nessa informação.

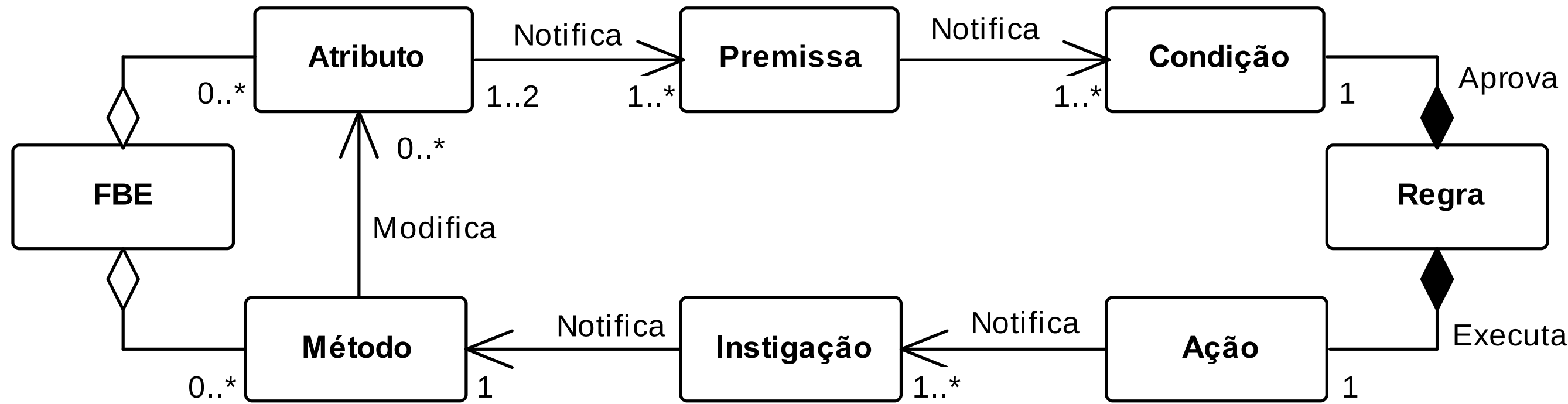


Fig. 1. Metamodelo do PON

O Elemento da Base de Fatos (Fact Base Element – **FBE**) armazena fatos do sistema por meio de **Atributos**. O FBE pode possuir **Métodos** que modificam esses Atributos. A **Premissa** é uma expressão causal relacionada a um Atributo que, quando notificada por ele, verifica se ele possui determinado valor (p.e. AtributoStatus == “Ligado”). A **Condição** possui uma ou mais Premissas associadas que, quando verdadeiras, aprovam uma **Regra**. A Regra realiza alguma ação quando é aprovada e, para isso, possui um elemento de **Ação**. Cada Ação irá notificar um conjunto de **Instigações** que, por sua vez, irá instigar um ou mais Métodos, modificando um ou mais Atributos. Esta sequência constitui o mecanismo de inferência do PON. Desse modo, aplicações em PON não dependem de um elemento centralizador, pois não necessitam de um motor de inferência.

## Materiais e métodos

O método DON se ocupou em: (i) estender diagramas da UML para representar os conceitos do PON - perfil PON, (ii) definir um método que use essa extensão em uma sequência de passos para conduzir o desenvolvedor na criação de aplicações PON.

Um perfil UML permite que a UML seja particularizada para um domínio específico de aplicações, possibilitando determinar uma nova sintaxe e semântica aos elementos existentes na UML. Para isso, faz uso de estereótipos, valores etiquetados e restrições. Os passos do método DON são ilustrados na Fig. 2.

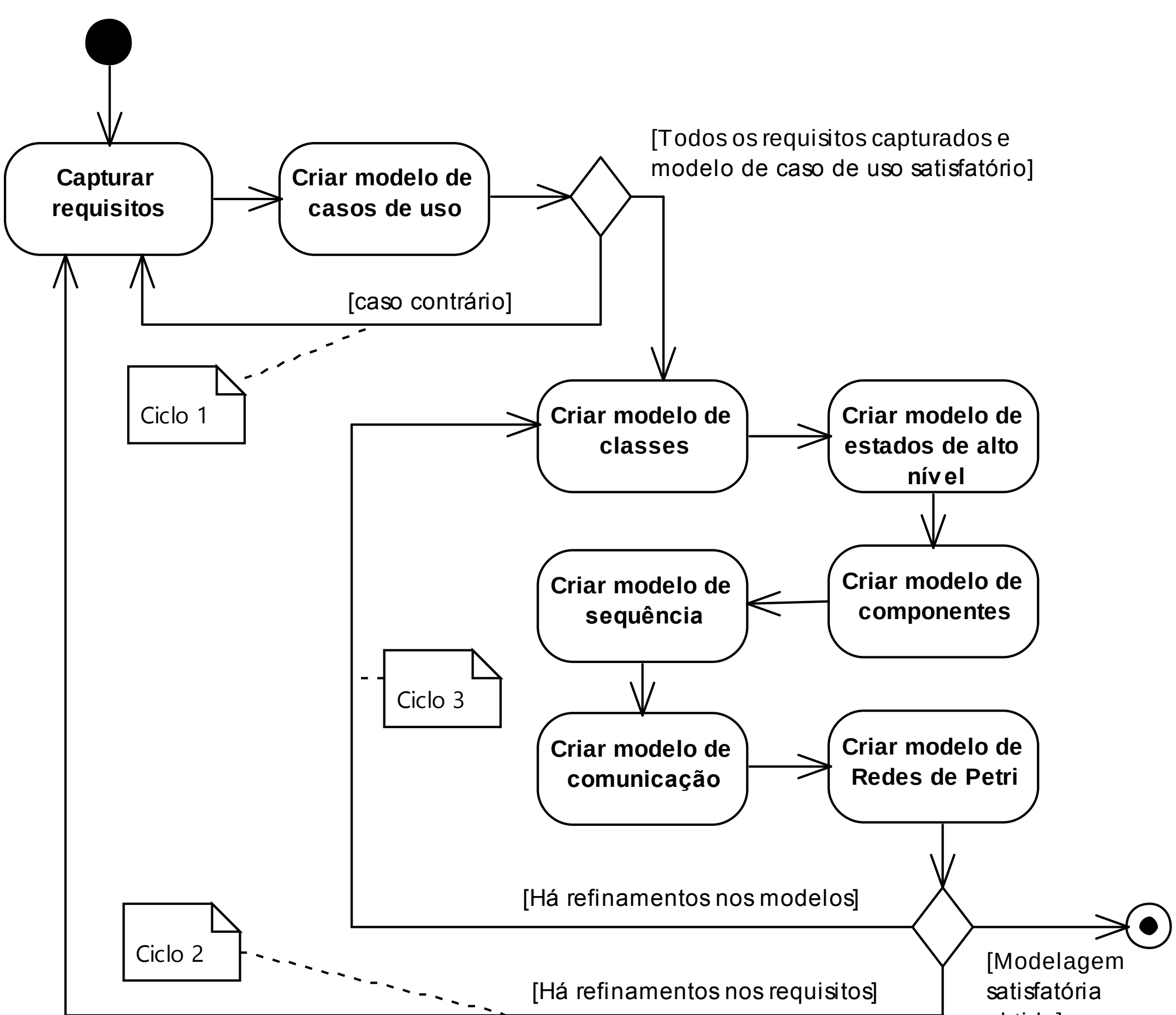


Fig. 2. Atividades e ciclos do DON

O método é dividido em três ciclos básicos. O primeiro compreende as atividades de levantamento de requisitos do software e é finalizado quando todos os requisitos foram capturados e tem-se um modelo de casos de uso satisfatório. O segundo ciclo cria as primeiras versões dos diagramas e retorna aos requisitos enquanto for necessário fazer refinamentos. Quando não houver mais refinamentos, o terceiro ciclo se concentra em finalizar os modelos até que os requisitos do software sejam atendidos. A codificação poderá ocorrer em ciclos intermediários, se o processo de software usado for iterativo e incremental, ou poderá iniciar ao final da aplicação do DON, caso o processo seja baseado no modelo de processo de software em cascata.

Um estudo de caso foi conduzido para avaliar a efetividade do método. Escolheu-se um jogo do tipo arcade, no qual o jogador controla um navio de guerra na parte inferior da tela atirando contra um avião de guerra inimigo na parte superior da tela do jogo. Alguns dos artefatos criados são ilustrados pelas Figs. 3 à 7 e Tab. 1.

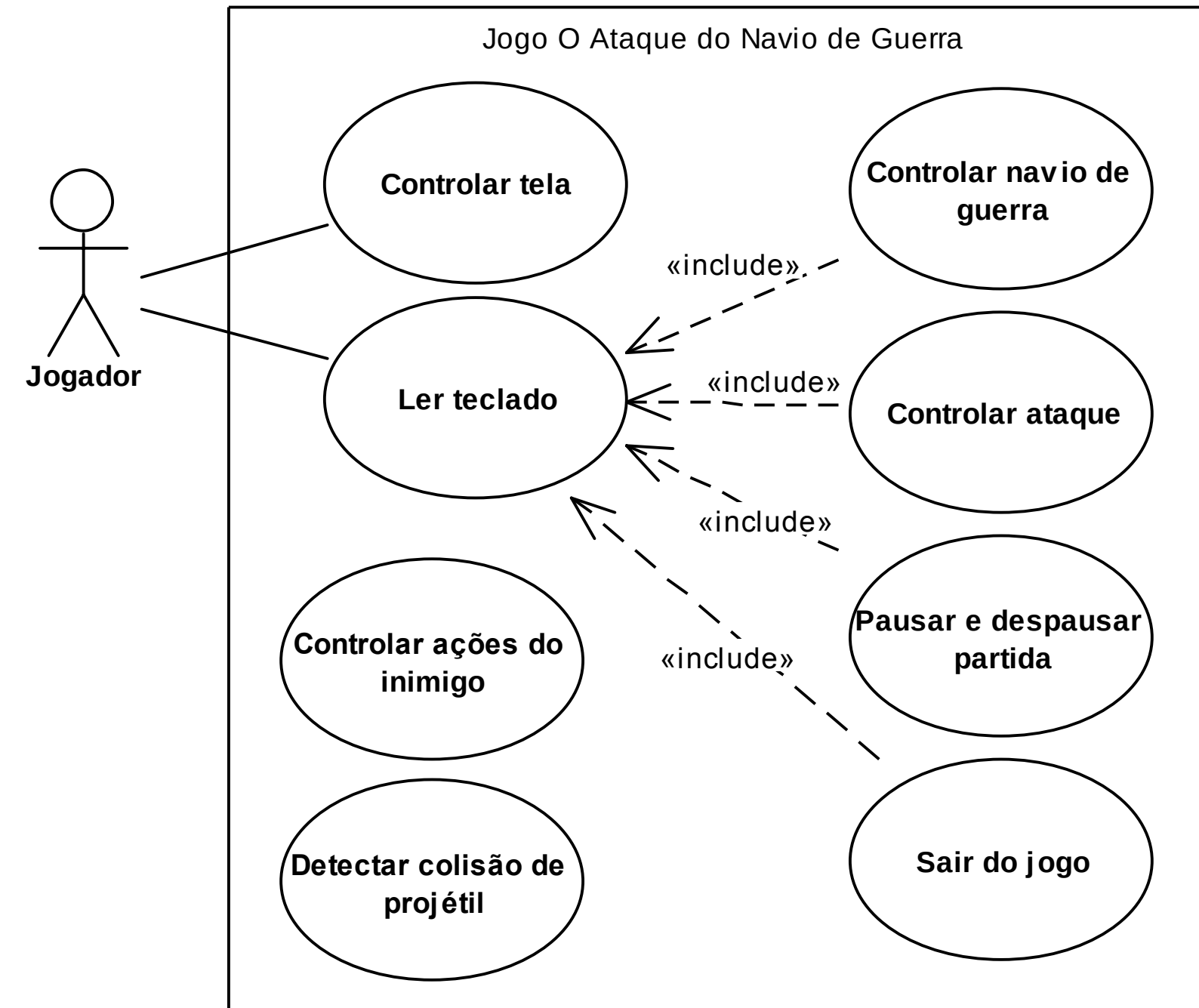


Fig. 3. Modelo de casos de uso.

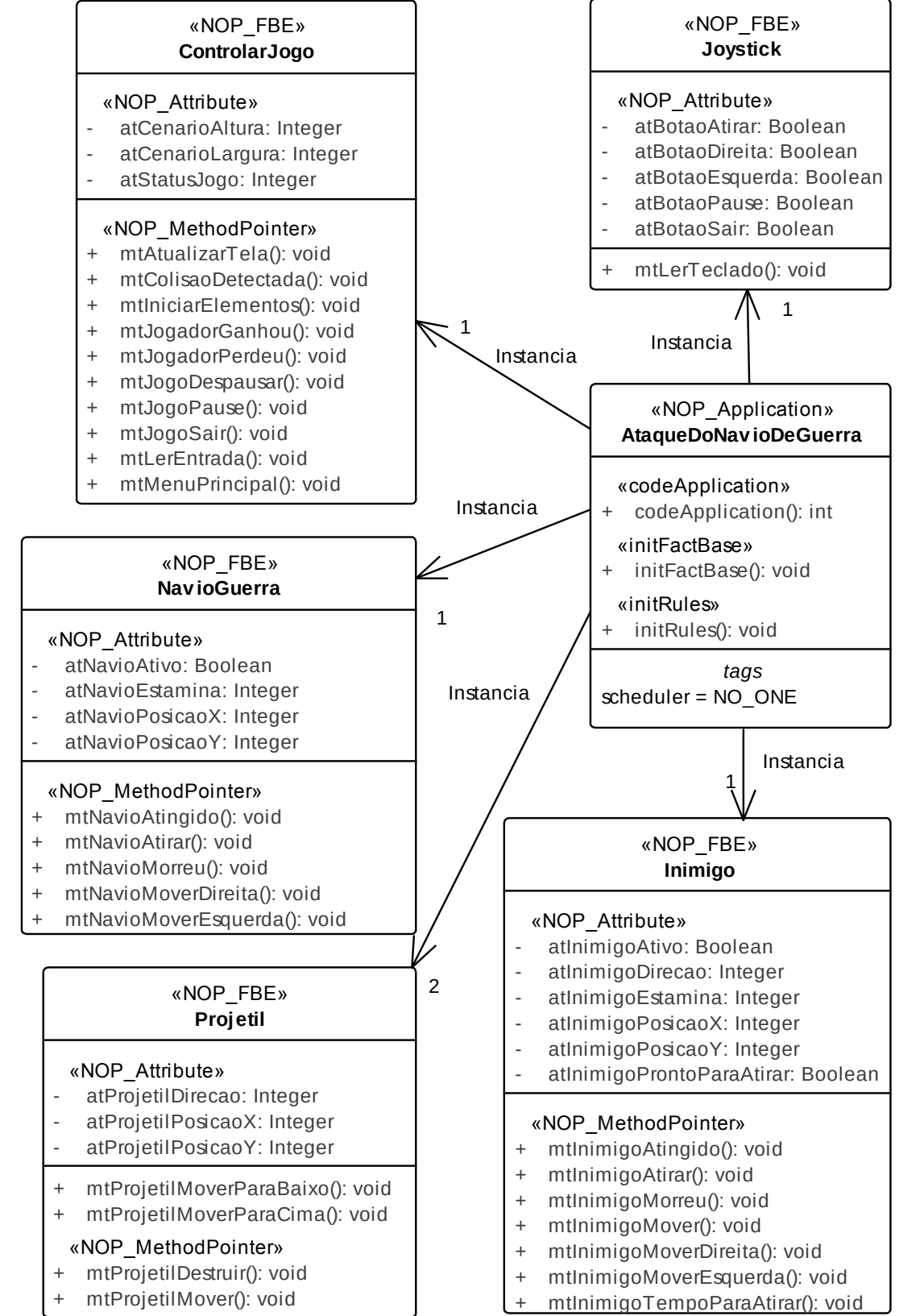


Fig. 4. Modelo de classes.

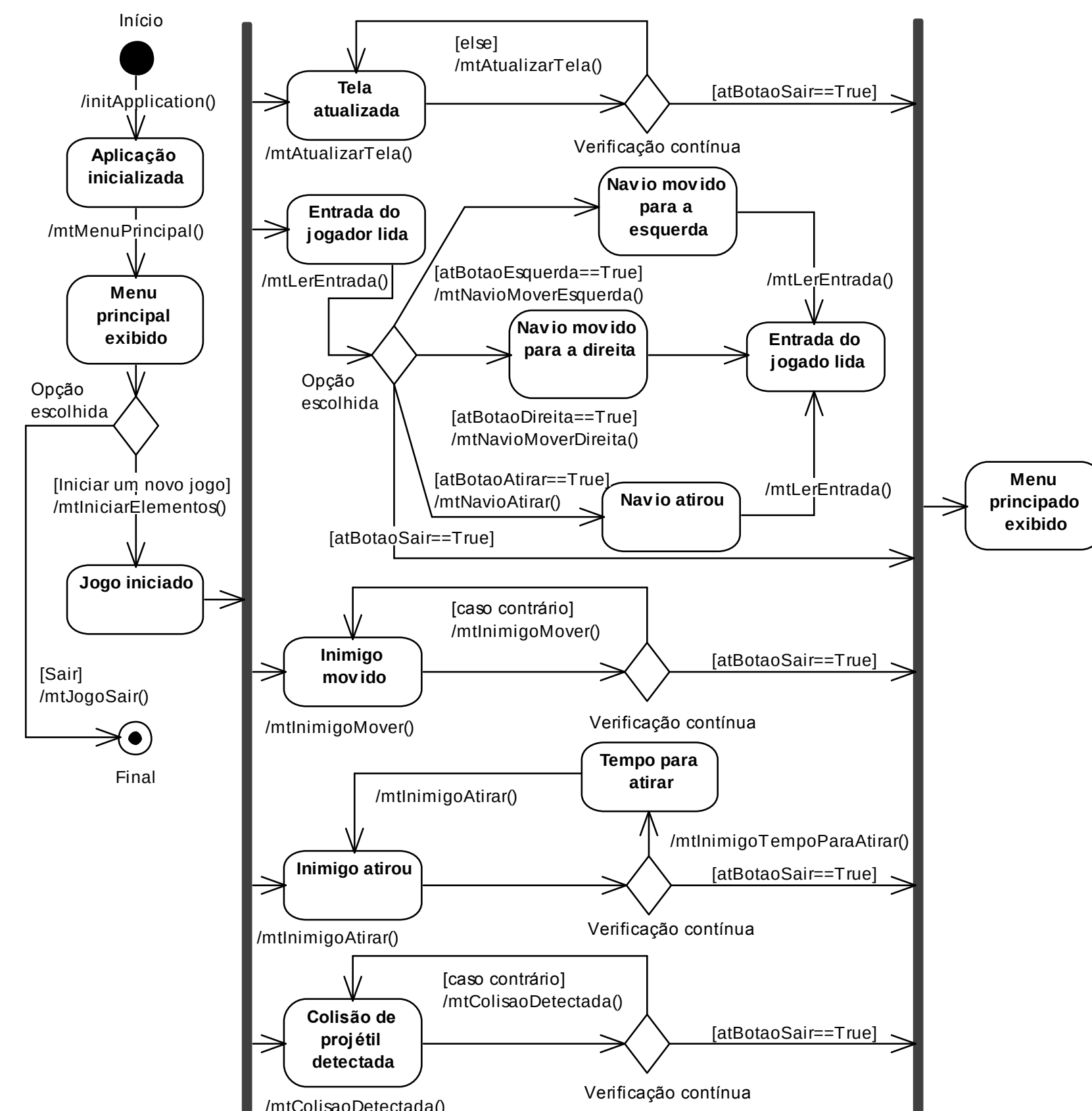


Fig. 5. Modelo de estados de alto nível.

| Regra | Nome                              |
|-------|-----------------------------------|
| 1     | rlAtualizarPosicaoObjetos         |
| 2     | rlInimigoMover                    |
| 3     | rlInimigoAtirar                   |
| 4     | rlInimigoModificarDirecaoDireita  |
| 5     | rlInimigoModificarDirecaoEsquerda |
| 6     | rlDetectarColisaoContraInimigo    |
| 7     | rlDetectarColisaoContraJogador    |
| 8     | rlNavioMoverEsquerda              |
| 9     | rlNavioMoverDireita               |
| 10    | rlNavioAtirar                     |
| 11    | rlJogoPausar                      |
| 12    | rlJogoDespausar                   |
| 13    | rlJogoParar                       |
| 14    | rlJogadorGanha                    |
| 15    | rlJogadorPerde                    |
| 16    | rlIniciarNovoJogo                 |

Tab. 1. Regras identificadas.

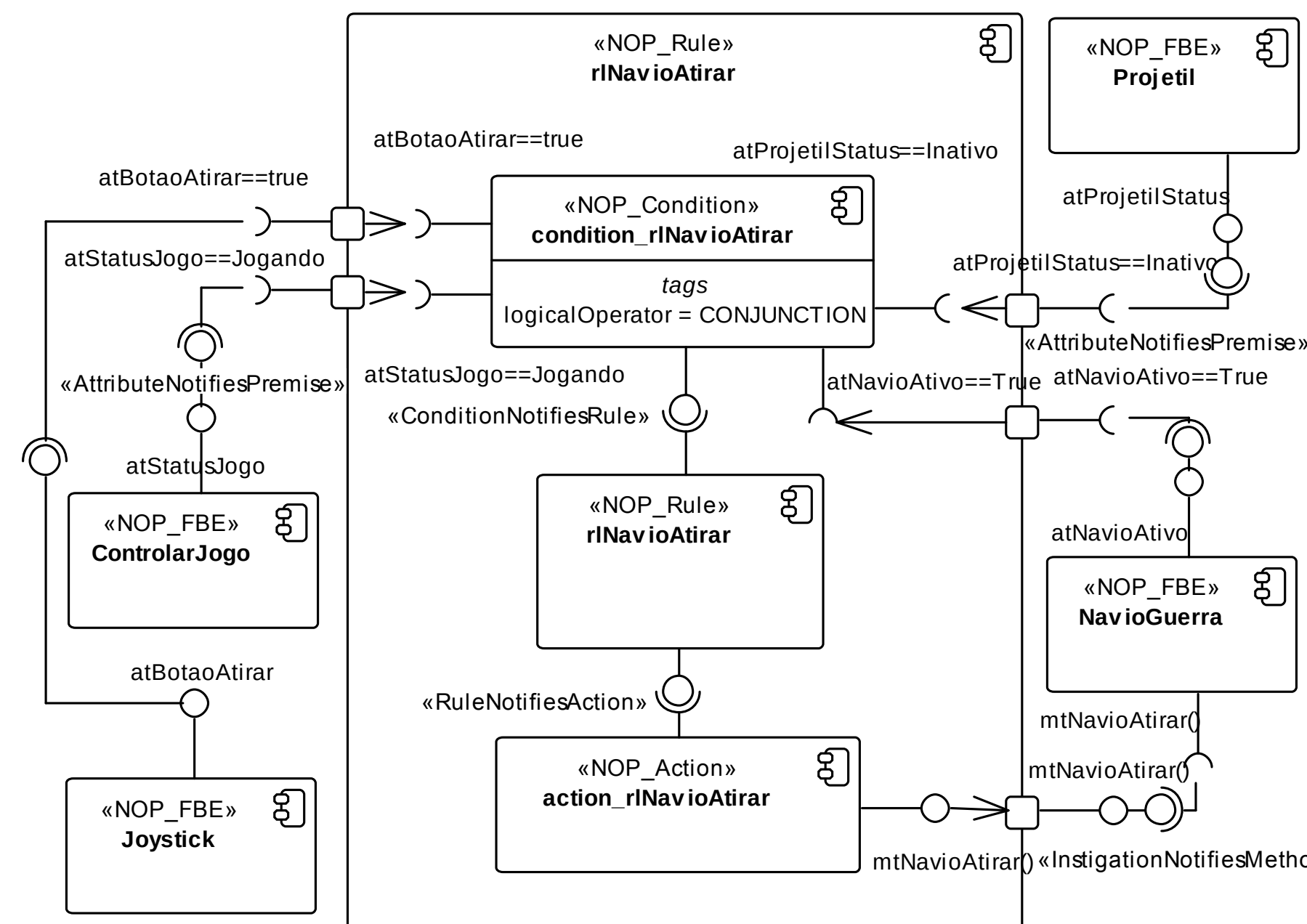


Fig. 6. Modelagem da regra rlNavioAtirar como componente.

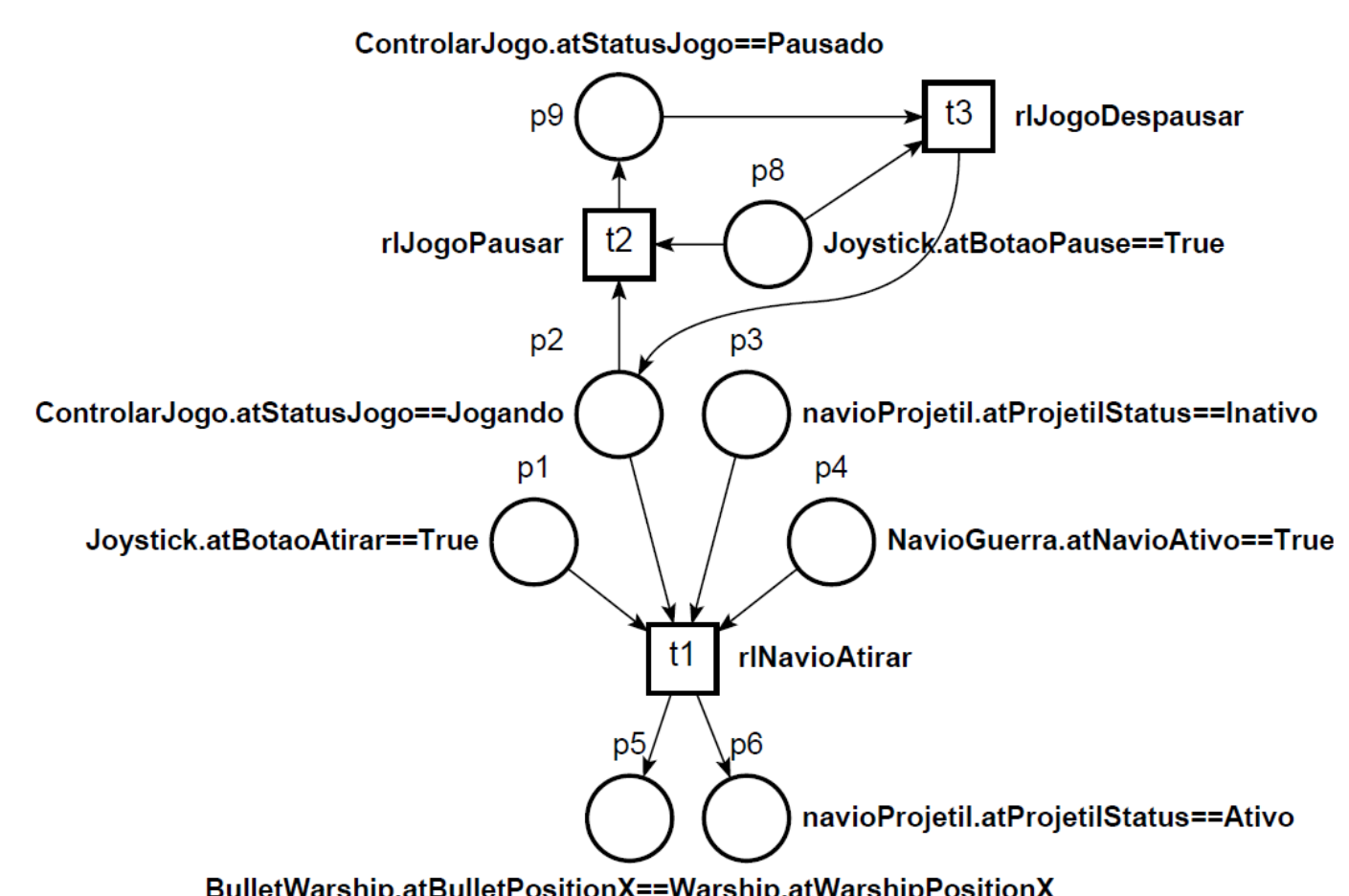


Fig. 7. Parte do Modelo de Rede de Petri.

## Resultados

O DON é iterativo, permitindo, assim, a melhoria dos modelos à medida que os ciclos ocorrem. Na utilização do DON para este trabalho, por exemplo, diversos atributos e métodos foram identificados somente durante a criação do modelo de componentes. Assim, os ciclos 2 e 3 do DON permitiram que os modelos anteriores fossem atualizados.

A existência de um perfil específico para o PON permitiu que a sintaxe e a semântica dos conceitos do PON fossem representados nos diagramas da UML. O modelo de Redes de Petri mostrou-se eficiente para simular o comportamento do software e permitir que ele seja comparado com os casos de uso e requisitos previamente estabelecidos.

## Conclusões

O uso do método DON supera algumas limitações da modelagem de softwares e fornece uma abordagem abrangente, fácil de usar e eficiente para desenvolvimento de softwares baseados no PON. No entanto, existem algumas lacunas. A análise realizada com UML não beneficia a descoberta das regras no PON. Em parte, isso se deve ao fato de que a UML não foi desenvolvida para este paradigma. Assim, o processo de identificação de regras torna-se um intenso processo de síntese e requer muito esforço do desenvolvedor.

Pesquisas do grupo, em andamento, incluem o uso de métodos alternativos de modelagem e a criação de uma linguagem e ferramentas específicas que possam aderir aos conceitos de Sistemas Orientados a Regras e, mais especificamente, ao Paradigma Orientado a Notificações. Tanto o DON quanto as novas proposições se aplicam, naturalmente, no âmbito de sistemas de inteligência computacional e artificial.